

ENGI 9839: Software Verification and Validation

Spring 2026

Project Proposal

TOPIC: Comparing Manual Unit Testing and Automated API Fuzzing for Data Integrity in a Django/PostgreSQL E-Commerce Backend

Team Members:

Olaleye Adeniyi Akinuli – Student ID: 202488212

Chiemerie Cletus Obijiaku – Student ID: 202492457

1. Introduction and Motivation

Software testing is one of the most actively debated areas in software engineering, particularly when it comes to understanding the trade-offs between human-authored tests and automated verification approaches. For this project, we propose to investigate that tension in a concrete, practical setting: a Django REST Framework and PostgreSQL e-commerce backend that we have already developed. Specifically, we intend to compare the effectiveness of manually written unit tests against automated API fuzzing powered by Schemathesis, an open-source fuzzing tool that operates against OpenAPI schemas.

The motivation behind this topic is straightforward: in real-world software teams, there is rarely a consensus on how much manual testing is sufficient, or whether automated tooling can reliably replace the judgment of a developer writing targeted test cases. By applying both approaches to the same system and the same endpoints, we hope to produce a side-by-side comparison that is grounded in evidence rather than preference. We believe this framing aligns well with the course's emphasis on evaluating verification techniques critically and rigorously.

2. Project Scope and Case Study

The subject of our project is an existing e-commerce web application consisting of a React frontend, a Django REST Framework API backend, and a PostgreSQL database. Rather than building a new system from scratch, we will use this application as a ready-made case study, which allows us to focus our effort on the verification work itself. We will narrow the scope of testing to one or two core API endpoints – most likely the order creation and cart update views — as these are sufficiently complex to exercise business logic, input validation, and database integrity constraints simultaneously.

3. Proposed Implementation Approach

The project will unfold across three phases:

Phase 1 – Manual Unit Testing

We will author a focused suite of some hand-written test cases using pytest-django and Django's built-in TestCase framework. Each test will be deliberately designed to probe a specific behaviour or edge case in the selected endpoints, such as boundary values, invalid

foreign key references, missing required fields, or authentication failures. Code coverage will be measured using coverage.py as a practical and widely used adequacy metric. We acknowledge that line coverage measures execution rather than fault detection – a known limitation in the literature – and will address this distinction explicitly in the report’s analysis, where coverage results will be interpreted alongside the qualitative nature of faults each approach was able to surface.

Phase 2 – Automated API Fuzzing

For the automated component, we will use drf-spectacular to generate an OpenAPI 3.0 schema from the existing DRF views, and then run Schemathesis against a locally hosted test server. Schemathesis will automatically generate thousands of requests containing random, malformed, and edge-case data – such as oversized strings, negative integers, null values, and unexpected field combinations – to identify crashes, unexpected 500 errors, or violations of database integrity constraints. We will configure a dedicated test database to isolate the fuzzing run from any development data.

Phase 3 – Analysis and Report

The final phase involves comparing the two approaches across a set of defined metrics: defects detected, code coverage achieved, setup effort, and the nature of faults found. We expect the comparison to reveal qualitative differences – for example, that manual tests are better suited to catching logic errors, while fuzzing excels at surfacing unhandled input validation failures. These findings will form the analytical core of the project report, supported by relevant literature.

4. Relevance to Course Themes

This project sits at the intersection of three topics highlighted in the course project guide suggested topic list. Most directly, it addresses comparing manual testing and automated verification approaches, which forms the central research question of the work. It also engages with the application of verification techniques to real-world case studies, given that the subject of analysis is an existing, functional e-commerce system rather than a purpose-built toy example. Finally, the project connects to themes of mutation testing and test adequacy assessment: while mutation testing is not the primary method employed, the question of what makes a test suite adequate – and how coverage-based metrics fall short of capturing fault-detection capability – is a thread that will run through the report’s analysis and discussion. We believe this breadth of connection reflects a considered and well-grounded choice of topic that is technically feasible within the project timeline, and capable of producing a substantive and evidence-based report.

Thank you for your consideration.